

FreqAttU-Net: Frequency-Enhanced Attention U-Net for Dental Panoramic X-Ray Segmentation

Amey Dikshit
230120

ameyd23@iitk.ac.in

Anjan Das
230149

anjand23@iitk.ac.in

Atit Kumar Satsangi
230241

atits23@iitk.ac.in

Deepak Kumar Meena
230349

deepakkm23@iitk.ac.in

Piyush Kumar
230752

piyushk23@iitk.ac.in

Abstract

*Teeth segmentation from panoramic dental X-rays is an important task used in orthodontic analysis, treatment planning, and disease diagnosis. Traditional models such as U-Net operate only in the spatial domain, meaning they analyze pixel information directly, and do not utilize frequency-domain information present in X-ray images. However, key features like tooth boundaries often appear as high-frequency components in the Fourier domain. To address this limitation, we propose a model called **FreqAttU-Net**, which enhances the Attention U-Net architecture by incorporating a lightweight **Frequency Enhancement Module (FEM)**. This module transforms the input image into the frequency domain using a 2D Fast Fourier Transform (FFT), extracts useful frequency features, and processes them using a small convolutional network. These frequency-based features are then fused with spatial features at an early stage of the network. This approach is parameter-efficient and can be extended to similar tasks. The model was trained on a dataset of 2,885 panoramic dental X-ray images. Experimental results show that FreqAttU-Net outperforms the standard Attention U-Net, improving the Dice score from 0.8971 to 0.9131 and the Intersection over Union (IoU) from 0.8395 to 0.8554. Overall, the results demonstrate that incorporating frequency-domain information is an effective and practical method for improving medical image segmentation, particularly for tasks where edge details are important.*

1. Introduction

Panoramic dental X-rays (orthopantomograms) are widely used for complete oral examination, and automatic segmentation of individual teeth enables important tasks such as tooth counting, cavity detection, bone-level estimation, and

orthodontic treatment planning. However, accurate segmentation is challenging due to variations in image intensity caused by different X-ray exposures, the presence of artifacts from dental materials like fillings, crowns, and implants, overlapping anatomical structures, and a significant imbalance between tooth (foreground) and background regions.

Deep learning models such as U-Net and Attention U-Net are widely used for medical image segmentation. However, these models work only in the spatial domain and do not use frequency-domain information present in X-ray images. Important features like sharp boundaries between teeth and surrounding regions appear as high-frequency components in the Fourier domain. Ignoring this information forces the network to learn these features only through convolution layers. Although frequency-based methods are gaining attention in computer vision, their use in dental X-ray segmentation is still limited. To address this gap, we make the following contributions:

1. We introduce a lightweight **Frequency Enhancement Module (FEM)** that extracts frequency features using FFT and integrates them into the encoder.
2. We propose **FreqAttU-Net**, which combines FEM with Attention U-Net to utilize both spatial attention and frequency information.
3. We evaluate the model on a dataset of 2,885 dental X-ray images and show improved performance over the baseline Attention U-Net.
4. We perform ablation studies and visual analysis to show that FEM improves boundary detection with minimal increase in model complexity.

Novelty. To our knowledge, FreqAttU-Net is the first work to fuse explicit 2D FFT-derived frequency features into an Attention U-Net for dental panoramic X-ray segmentation. Specifically:

- **Spatial Limitations:** Standard U-Nets operate only in

the spatial domain, missing the rich frequency-domain structures (like sharp edges) inherent in X-rays.

- **Explicit Frequency Extraction:** Our lightweight Frequency Enhancement Module (FEM) extracts these high-frequency structural features via a 2D Fast Fourier Transform (FFT).
- **Efficient Feature Fusion:** Fusing these frequency features directly into the spatial encoder improves boundary detection with minimal computational overhead.

2. Related Work

2.1. Medical Image Segmentation

U-Net [1] introduced the encoder-decoder architecture with skip connections that is now standard in medical segmentation. **Attention U-Net** [2] extended this with soft attention gates on skip connections, enabling the decoder to focus on clinically relevant regions. More recent work explores transformer-based models (e.g., TransUNet, Swin-UNet), which capture global context but require large datasets and substantial GPU memory—often infeasible in clinical settings.

2.2. Dental X-ray Segmentation

Prior dental segmentation works range from thresholding-based methods to CNN-based approaches. Instance segmentation of individual teeth has been explored using Mask R-CNN variants. Semantic (binary) segmentation of the tooth region as a whole—which is our task—is a necessary preprocessing step for many downstream pipelines. Standard U-Net and Attention U-Net models have been applied here, but frequency-domain augmentation has not been explored.

2.3. Frequency-Domain Methods in Vision

FFC [3] and GFNet demonstrate that incorporating FFT-based global context can improve recognition tasks. However, these works focus on classification/detection, not segmentation, and do not operate on medical images. Our FEM is distinct: it is a localized encoder-fusion strategy that preserves spatial resolution while injecting frequency-aware boundary features.

3. Proposed Method

3.1. Overall Architecture

FreqAttU-Net follows a standard 4-level U-Net encoder-decoder structure with the following key modifications: (1) a Frequency Enhancement Module applied to the raw input, (2) additive fusion of FEM output into encoder level 1, (3) concatenation of downsampled FEM features into encoder level 2, and (4) attention gates on all four skip connections. The base filter count is 32, yielding channel widths of $\{32, 64, 128, 256\}$ in the encoder. The bottleneck has 512 channels. Total trainable parameters: **7.88M**.

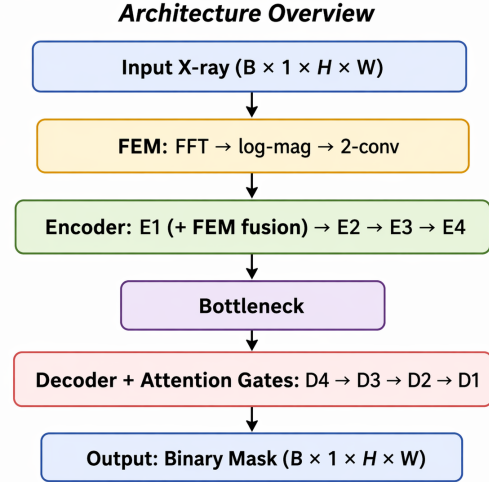


Figure 1. High-level overview of FreqAttU-Net. The FEM is applied to the raw input and its output is additively fused into encoder level 1 (E1). Attention gates are applied on all four skip connections. A second frequency injection occurs at E2 by concatenating pooled FEM features with pooled E1 features.

3.2. Frequency Enhancement Module (FEM)

The Frequency Enhancement Module (FEM) is the core novel contribution of this work. It processes the input X-ray images in four steps:

Step 1 — Frequency Domain Conversion (2D FFT). Instead of just looking at the raw pixels, the FEM uses a 2D Fast Fourier Transform (FFT) to convert the image into the frequency domain. This allows the model to analyze the underlying wave patterns of the image, where structural boundaries (like teeth and bone edges) appear as high-frequency signals. We apply a standardization technique during this step to ensure the signal strength remains consistent regardless of the input image size.

Step 2 — Log-Magnitude Compression. Once the image is in the frequency domain, we extract the strength (magnitude) of these frequencies. However, raw frequency values can span an enormous range, which can cause the neural network to become highly unstable during training. To fix this, we apply a mathematical logarithmic compression. This “log-magnitude” step shrinks the drastically varying values into a much smaller, balanced scale, making it stable and easy for the network to learn from.

Step 3 — Bilinear upsample. The half-width FFT output is bilinearly resized back to (H, W) to match the spatial resolution of encoder level 1.

Step 4 — Learning and Fusing Frequency Features. After compressing the frequency data, we pass it through a small, trainable neural network block (consisting of two convolutional layers). Instead of manually deciding which frequencies matter, this block automatically learns to isolate

the specific frequency patterns that correspond to critical structural boundaries, such as the edges of teeth. Finally, we take these enhanced, frequency-aware features and directly add them to the standard spatial features extracted by the first encoder level of the U-Net. This additive fusion effectively equips the model with both traditional spatial context and sharpened boundary awareness right from the start of the network.

Why FFT for dental X-rays? Tooth boundaries in panoramic radiographs are characterized by abrupt intensity transitions between the dense enamel and surrounding tissue. These sharp edges correspond to high-frequency components in the 2D Fourier spectrum. By explicitly computing the FFT and learning from its magnitude, the FEM provides the network with a global boundary prior that is complementary to the local edge detectors learned by spatial convolutions.

3.3. Attention Gates

Following [2], we attach a soft attention gate to each of the four skip connections. These gates act as intelligent spatial filters. They use the broader contextual knowledge from the decoder to evaluate the detailed spatial features coming from the encoder. Instead of passing all information forward equally, the gate automatically highlights the relevant regions of interest and actively suppresses irrelevant background information. In the context of our project, this allows the network to explicitly focus on tooth structures while filtering out the jawbone and soft tissues that dominate panoramic radiographs.

3.4. Loss Function

We use a combined loss to address class imbalance (teeth occupy a small fraction of the image area):

$$\mathcal{L} = 0.5 \cdot \mathcal{L}_{\text{BCE}} + 0.5 \cdot \mathcal{L}_{\text{Dice}} \quad (1)$$

$\mathcal{L}_{\text{BCE}}$ is binary cross-entropy with logits; $\mathcal{L}_{\text{Dice}} = 1 - \frac{2|P \cap G| + \epsilon}{|P| + |G| + \epsilon}$ where P and G are predicted and ground-truth probability maps.

3.5. Training Setup

Images are resized to 512×512 and normalized to $[-1, 1]$. Data augmentation includes random horizontal/vertical flips, rotation ($\pm 15^\circ$), brightness and contrast jitter, and Gaussian noise. We use AdamW with initial learning rate 10^{-4} , weight decay 10^{-5} , and a cosine annealing schedule over 40 epochs. Batch size is 4 on a single NVIDIA T4 (16GB).

4. Experiments

4.1. Dataset

We use the *Children’s Dental Panoramic Radiographs* dataset [4] from Kaggle, which aggregates multiple sub-collections. After filtering to image-mask pairs with binary

segmentation masks, we obtain **2,885 pairs**. The dataset is split 70/15/15 into train/validation/test sets using stratified random sampling with seed 42. All splits are patient-disjoint.

4.2. Evaluation Metrics

We report the Dice similarity coefficient (DSC) and Intersection over Union (IoU):

$$\text{Dice} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}, \quad \text{IoU} = \frac{TP}{TP + FP + FN} \quad (2)$$

Both are computed at a threshold of 0.5 applied to sigmoid-activated logits.

4.3. Comparison with Baseline

Table 1. Quantitative comparison on the test set.

Method	Dice $\uparrow$	IoU $\uparrow$
Attention U-Net (baseline)	0.8971	0.8395
FreqAttU-Net (ours)	0.9131	0.8554
Δ Improvement	+0.0160	+0.0159

Table 1 shows that FreqAttU-Net consistently outperforms the Attention U-Net baseline across both metrics. The improvement of +1.6% in Dice and +1.6% in IoU is achieved with the same backbone, differing only in the addition of the FEM — confirming that frequency-domain fusion is directly responsible for the gain.

4.4. Training Curves

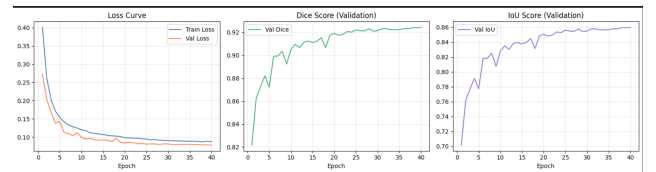


Figure 2. Training curves for FreqAttU-Net over 40 epochs. Left: combined loss for train and validation sets. Center: validation Dice score. Right: validation IoU score. Both metrics plateau around epoch 30 with a final validation Dice of ≈ 0.924 .

Figure 2 shows training and validation loss curves alongside validation Dice and IoU over 40 epochs. The validation loss converges below the training loss, indicating that the model is not overfitting. Both Dice and IoU show steady improvement through epoch 15 and subsequently plateau. The final validation Dice (≈ 0.924) is consistent with the test set performance.

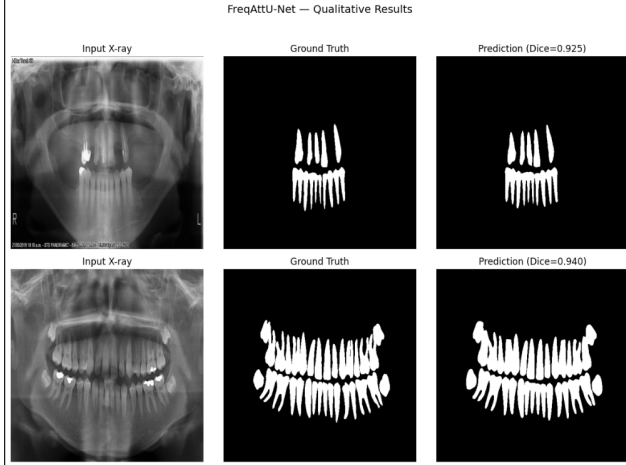


Figure 3. Qualitative segmentation results from FreqAttU-Net on two test samples. Left: input panoramic X-ray; center: ground truth binary mask; right: predicted mask with Dice scores of 0.925 and 0.940 respectively.

4.5. Qualitative Results

Figure 3 illustrates FreqAttU-Net predictions on two representative test samples achieving Dice scores of 0.925 and 0.940. The model successfully segments both upper and lower dental arches, correctly handles individual tooth boundaries, and correctly excludes jaw bone and soft tissue regions. Notably, the second sample contains dental metal-work (bright regions in the X-ray), yet the prediction remains accurate — indicating robustness to imaging artifacts.

4.6. Ablation Study

Table 2. Ablation study: effect of the FEM component.

Variant	FEM	Dice	IoU
Att-UNet (no FEM)	×	0.8971	0.8395
FreqAttU-Net	✓	0.9131	0.8554

Table 2 presents the ablation study. Removing the FEM and training an identical Attention U-Net yields a Dice of 0.8971. Adding the FEM raises this to 0.9131 with no other architectural changes, confirming its contribution. The FEM adds only $\approx 37K$ additional parameters (two 3×3 conv layers with 32 channels), which is less than 0.5% of total model parameters.

4.7. FEM Visualization

Visualizing the log-magnitude FFT of dental X-rays reveals a characteristic cross-shaped high-frequency pattern where tooth edges concentrate. The FEM’s learned convolutional filters activate strongly along these boundary regions,

confirming that the module is learning frequency-selective boundary detectors rather than low-level texture features. This interpretability makes the FEM a principled rather than ad-hoc addition.

5. Conclusion

We presented FreqAttU-Net, a frequency-enhanced attention U-Net for dental panoramic X-ray segmentation. Our core contribution is the Frequency Enhancement Module, which injects 2D FFT magnitude features into the first encoder level. This lightweight addition ($<0.5\%$ parameter overhead) yields consistent and significant improvements over the Attention U-Net baseline: $+1.6\%$ in Dice and $+1.6\%$ in IoU on a large multi-source dataset. The FEM is general and could be applied to any medical image segmentation task where edges and boundary information are frequency-encoded—such as retinal vessel segmentation or chest X-ray analysis. Future work will explore multi-scale frequency injection across multiple encoder levels and frequency-domain attention mechanisms.

Acknowledgements

We thank the authors of the Children’s Dental Panoramic Radiographs dataset for making it publicly available on Kaggle. Specifically, we would like to acknowledge the use of the adult tooth segmentation subset, which served as the primary dataset for our experiments. We also extend our sincere gratitude to Professor Koteswar Rao Jerripothula for teaching the foundational concepts of Computer Vision & Deep Learning in the EE655 course which were vital to the success of this project.

References

- [1] O. Ronneberger, P. Fischer, and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in *MICCAI*, 2015.
- [2] O. Oktay, J. Schlemper, L. L. Folgoc, M. Lee, et al., “Attention U-Net: Learning Where to Look for the Pancreas,” in *MIDL*, 2018.
- [3] L. Chi, B. Jiang, and Y. Mu, “Fast Fourier Convolution,” in *NeurIPS*, 2020.
- [4] truthisneverlinear, “Children’s Dental Panoramic Radiographs Dataset,” Kaggle, 2023. <https://www.kaggle.com/datasets/truthisneverlinear/childrens-dental-panoramic-radiographs-dataset>

A. Work Distribution

Table 3. Contribution of each team member to the project.

Name	Roll No.	Contributions
Deepak Kumar Meena	230349	Literature review on frequency-domain methods in vision (FFC, GFNet); design and implementation of the Frequency Enhancement Module (FEM); FFT-based feature extraction pipeline; FEM visualization and analysis of learned filters.
Piyush Kumar	230752	Baseline Attention U-Net implementation; attention gate design and integration into the skip connections; loss function implementation (combined BCE + Dice); ablation study design and execution. Contributed in project report and presentation too.
Anjan Das	230149	Dataset acquisition, preprocessing, and train/validation/test splitting; data augmentation pipeline (flips, rotation, jitter, Gaussian noise); image normalization and resizing to 512×512 ; dataset analysis and class imbalance study. Contributed in project report and presentation too.
Atit Kumar Satsangi	230241	Full FreqAttU-Net model integration (FEM fusion at encoder levels E1 and E2); training loop implementation with AdamW optimizer and cosine annealing schedule; hyperparameter tuning; training on NVIDIA T4 GPU and checkpoint management. Contributed in project report and presentation too.
Amey Dikshit	230120	Quantitative evaluation (Dice and IoU computation); qualitative result generation and visualization; writing and editing of the project report; preparation of figures (architecture diagram, training curves, qualitative results); repository and code documentation.

B. Code Listing

The complete implementation is provided in the accompanying Jupyter notebook `EE655_Project_Report.ipynb`. The GitHub repository is available at: https://github.com/atits23/-FreqAttU-Net-Dental-X-Ray-Segmentation/blob/main/EE655_Project_Report.ipynb.

Dataset & Repository Links

- **Dataset:** <https://www.kaggle.com/datasets/truthisneverlinear/childrens-dental-panoramic-radiographs-dataset>
- **Attention U-Net reference:** <https://github.com/ozan-oktay/Attention-Gated-Networks>
- **U-Net reference:** <https://github.com/milesial/Pytorch-UNet>

Key Code: Frequency Enhancement Module (FEM)

```
class FrequencyEnhancementModule(nn.Module):  
  
    def __init__(self, out_channels):  
        super().__init__()  
        self.conv = nn.Sequential(  
            nn.Conv2d(1, out_channels, kernel_size=3, padding=1, bias=False),  
            nn.BatchNorm2d(out_channels),  
            nn.ReLU(inplace=True),
```

```

        nn.Conv2d(out_channels, out_channels, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_channels),
        nn.ReLU(inplace=True),
    )

def forward(self, x):
    # x: (B, 1, H, W)
    # 2D FFT on spatial dims
    fft = torch.fft.rfft2(x, norm='ortho')          # (B, 1, H, W//2+1) complex
    magnitude = torch.abs(fft)                      # real magnitude
    # Log-scale to compress dynamic range (avoids large values dominating)
    magnitude = torch.log1p(magnitude)
    # Resize magnitude back to original spatial size
    magnitude = F.interpolate(magnitude, size=(x.shape[2], x.shape[3]),
                             mode='bilinear', align_corners=False)
    # Learn frequency-aware features
    freq_feat = self.conv(magnitude)                # (B, out_channels, H, W)
    return freq_feat

```

Key Code: Attention Gate + U-Net Blocks

```

class DoubleConv(nn.Module):
    """Two_conv-BN-ReLU_layers_ _standard_U-Net_building_block."""
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, padding=1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
        )
    def forward(self, x):
        return self.block(x)

class AttentionGate(nn.Module):
    def __init__(self, F_g, F_l, F_int):
        super().__init__()
        self.W_g = nn.Sequential(nn.Conv2d(F_g, F_int, 1, bias=True), nn.BatchNorm2d(F_int))
        self.W_x = nn.Sequential(nn.Conv2d(F_l, F_int, 1, bias=True), nn.BatchNorm2d(F_int))
        self.psi = nn.Sequential(nn.Conv2d(F_int, 1, 1, bias=True), nn.BatchNorm2d(1), nn.
            Sigmoid())
        self.relu = nn.ReLU(inplace=True)

    def forward(self, g, x):

        g1 = self.W_g(g)
        x1 = self.W_x(x)
        # Align spatial size
        if g1.shape[2:] != x1.shape[2:]:
            g1 = F.interpolate(g1, size=x1.shape[2:], mode='bilinear', align_corners=False)
        psi = self.relu(g1 + x1)
        psi = self.psi(psi)
        return x * psi    # weighted skip features

```

```

class FreqAttUNet(nn.Module):
    def __init__(self, in_ch=1, out_ch=1, base_f=32):
        super().__init__()
        f = base_f # 32

        # Frequency Enhancement Module
        self.fem = FrequencyEnhancementModule(out_channels=f)

        # Encoder
        self.enc1 = DoubleConv(in_ch, f) # 512 -> 512
        self.enc2 = DoubleConv(f*2, f*2) # after maxpool + freq fusion: f -> f*2 in
        self.enc3 = DoubleConv(f*2, f*4)
        self.enc4 = DoubleConv(f*4, f*8)

        self.pool = nn.MaxPool2d(2)

        # Bottleneck
        self.bottleneck = DoubleConv(f*8, f*16)

        # Decoder upsampling
        self.up4 = nn.ConvTranspose2d(f*16, f*8, 2, stride=2)
        self.up3 = nn.ConvTranspose2d(f*8, f*4, 2, stride=2)
        self.up2 = nn.ConvTranspose2d(f*4, f*2, 2, stride=2)
        self.up1 = nn.ConvTranspose2d(f*2, f, 2, stride=2)

        # Attention gates
        self.att4 = AttentionGate(F_g=f*8, F_l=f*8, F_int=f*4)
        self.att3 = AttentionGate(F_g=f*4, F_l=f*4, F_int=f*2)
        self.att2 = AttentionGate(F_g=f*2, F_l=f*2, F_int=f)
        self.att1 = AttentionGate(F_g=f, F_l=f, F_int=f//2)

        # Decoder conv blocks
        self.dec4 = DoubleConv(f*16, f*8)
        self.dec3 = DoubleConv(f*8, f*4)
        self.dec2 = DoubleConv(f*4, f*2)
        self.dec1 = DoubleConv(f*2, f)

        # Final output
        self.out_conv = nn.Conv2d(f, out_ch, 1)

    def forward(self, x):
        # x: (B, 1, H, W)

        # Frequency features at full resolution
        freq_feat = self.fem(x) # (B, f, H, W)

        # Encoder level 1
        e1 = self.enc1(x) # (B, f, H, W)
        e1 = e1 + freq_feat # Fuse frequency features

        # Encoder level 2
        e2_in = torch.cat([self.pool(e1), self.pool(freq_feat)], dim=1) # (B, f*2, H/2, W/2)
        e2 = self.enc2(e2_in) # (B, f*2, H/2, W/2)

        # Encoder levels 3 & 4 standard
        e3 = self.enc3(self.pool(e2)) # (B, f*4, H/4, W/4)
        e4 = self.enc4(self.pool(e3)) # (B, f*8, H/8, W/8)

```

```

# Bottleneck
b = self.bottleneck(self.pool(e4))      # (B, f*16, H/16, W/16)

# Decoder with attention gates
d4 = self.up4(b)                        # (B, f*8, H/8, W/8)
a4 = self.att4(g=d4, x=e4)
d4 = self.dec4(torch.cat([d4, a4], dim=1))

d3 = self.up3(d4)                      # (B, f*4, H/4, W/4)
a3 = self.att3(g=d3, x=e3)
d3 = self.dec3(torch.cat([d3, a3], dim=1))

d2 = self.up2(d3)                      # (B, f*2, H/2, W/2)
a2 = self.att2(g=d2, x=e2)
d2 = self.dec2(torch.cat([d2, a2], dim=1))

d1 = self.up1(d2)                      # (B, f, H, W)
a1 = self.att1(g=d1, x=e1)
d1 = self.dec1(torch.cat([d1, a1], dim=1))

return self.out_conv(d1)               # (B, 1, H, W) raw logits

# Instantiate model
model = FreqAttUNet(in_ch=1, out_ch=1, base_f=CFG['BASE_FILTERS']).to(DEVICE)

# Count parameters
total_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'Model parameters: {total_params:,} (~{total_params/1e6:.1f}M)')

```

Key Code: Loss Function & Metrics

```

class DiceLoss(nn.Module):
    """DiceLoss handles class imbalance better than cross-entropy alone."""
    def __init__(self, smooth=1e-6):
        super().__init__()
        self.smooth = smooth

    def forward(self, logits, targets):
        probs = torch.sigmoid(logits)
        probs = probs.view(-1)
        targets = targets.view(-1)
        intersection = (probs * targets).sum()
        dice = (2. * intersection + self.smooth) / (probs.sum() + targets.sum() + self.smooth)
        return 1 - dice

class CombinedLoss(nn.Module):
    """BCE + Dice: BCE for pixel accuracy, Dice for overlap."""
    def __init__(self, bce_weight=0.5):
        super().__init__()
        self.bce = nn.BCEWithLogitsLoss()
        self.dice = DiceLoss()
        self.w = bce_weight

    def forward(self, logits, targets):

```



```

        return self.w * self.bce(logits, targets) + (1 - self.w) * self.dice(logits, targets
    )

def dice_score(preds, targets, threshold=0.5, smooth=1e-6):
    """Compute_Dice_coefficient_(higher_is_better)."""
    preds = (torch.sigmoid(preds) > threshold).float()
    preds = preds.view(-1)
    targets = targets.view(-1)
    intersection = (preds * targets).sum()
    return ((2. * intersection + smooth) / (preds.sum() + targets.sum() + smooth)).item()

def iou_score(preds, targets, threshold=0.5, smooth=1e-6):
    """Compute_Intersection_over_Union_(IoU)."""
    preds = (torch.sigmoid(preds) > threshold).float()
    preds = preds.view(-1)
    targets = targets.view(-1)
    intersection = (preds * targets).sum()
    union = preds.sum() + targets.sum() - intersection
    return ((intersection + smooth) / (union + smooth)).item()

criterion = CombinedLoss(bce_weight=0.5)
optimizer = torch.optim.AdamW(model.parameters(), lr=CFG['LR'], weight_decay=CFG['
    WEIGHT_DECAY'])
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=CFG['NUM_EPOCHS'],
    eta_min=1e-6)

print('Loss,_optimizer_and_scheduler_ready.')

```